



ITESO, Universidad
Jesuita de Guadalajara

TAREA 2: Reporte de práctica 3 (Algoritmos Genéticos)

Integrantes:

- Valeria Esmer Lobo
- Dayanni Godoy Rosales
- Estephany Lisset Herrera Placencia
- Maria Fernanda Vargas Vaca

Fecha: 09 de Septiembre de 2026

Institución: Instituto Tecnológico y de Estudios Superiores de Occidente

Materia: Métodos de Inteligencia Artificial

Índice:

- [Introducción](#)
- [Objetivos](#)

- 1. Definición del Problema
- 2. Descripción del Algoritmo
- 3. Pruebas y Resultados
- Análisis de Resultados de la Población Final
- Conclusiones
- Conclusiones personales

```
In [1]: #Librerías
import numpy as np
import matplotlib.pyplot as plt
import time
import pandas as pd
```

Introducción

Los algoritmos genéticos son métodos de optimización inspirados en los principios de la evolución natural y la selección genética. Estos algoritmos trabajan con una población de posibles soluciones que, a través de diferentes procesos como selección, cruce y mutación, van evolucionando durante varias generaciones con el objetivo de encontrar soluciones cada vez mejores al problema planteado.

En esta práctica se implementa un Algoritmo Genético con codificación binaria aplicado a la función **Rastrigin** en dos dimensiones. Esta función es utilizada frecuentemente para evaluar algoritmos de optimización debido a que presenta múltiples máximos y mínimos locales, lo que hace más difícil encontrar la solución óptima mediante métodos tradicionales. Por esta razón, Rastrigin permite analizar la capacidad del algoritmo genético para explorar diferentes regiones del espacio de búsqueda y evitar quedar atrapado en soluciones locales.

Aunque la función Rastrigin normalmente se presenta como un problema de minimización, en este trabajo se plantea como un problema de maximización, por lo que el algoritmo busca encontrar el individuo que produzca el mayor valor de la función dentro del dominio establecido.

Objetivos

Diseñar e implementar un Algoritmo Genético de codificación binaria que permita encontrar el máximo de la función Rastrigin en dos dimensiones, utilizando los procesos de selección, cruce y mutación para evolucionar una población de soluciones a través de diferentes generaciones.

1. Definición del Problema

El problema consiste en implementar un Algoritmo Genético de codificación binaria capaz de encontrar el máximo de la función Rastrigin en dos dimensiones.

La función utilizada es:

$$f(x_1, x_2) = 20 + x_1^2 - 10 \cos(2\pi x_1) + x_2^2 - 10 \cos(2\pi x_2)$$

donde:

$$-5.12 \leq x_1, x_2 \leq 5.12$$

El reto principal consiste en encontrar, mediante el proceso evolutivo, los valores de (x_1) y (x_2) que produzcan el mayor valor posible de la función dentro del intervalo establecido.

Para lograrlo, cada solución se representa mediante una cadena binaria. En este caso, se utilizan 28 bits en total, que posteriormente son divididos para representar las dos variables. Estas representaciones binarias son transformadas a valores reales dentro del dominio de la función. El algoritmo debe mantener una población de soluciones y mejorarla progresivamente mediante:

Selección: elegir los individuos con mejor desempeño. Cruce: combinar información genética de dos individuos para generar nuevos candidatos. Mutación: modificar aleatoriamente un bit para introducir diversidad. Evaluación: medir qué tan buena es cada solución mediante la función Rastrigin.

De esta manera, el problema de optimización puede plantearse como:

$$\boxed{\max f(x_1, x_2)}$$

sujeto a:

$$-5.12 \leq x_1, x_2 \leq 5.12$$

2. Descripción del Algoritmo

El código implementa un Algoritmo Genético de codificación binaria diseñado para **maximizar** la función Rastrigin 2D. Utiliza selección por truncamiento (conserva la mejor mitad de la población, elitismo), cruce de un punto aleatorio a lo largo del cromosoma (28 bits totales) y una tasa de mutación baja a nivel global.

Pseudocódigo:

1. INICIO
2. Inicializar población aleatoria X_1, X_2 de tamaño N .
3. Mapear enteros binarios a números reales en el dominio $[-5.12, 5.12]$.
4. PARA cada generación HASTA G :

5. Evaluar función objetivo (Rastrigin).
6. Ordenar de menor a mayor desempeño.
7. Seleccionar la mitad superior (mejores $N/2$ padres para maximizar).
8. Cruzar padres seleccionados mediante un punto de cruce aleatorio para generar $N/2$ hijos.
9. Mutar: Con 10% de probabilidad, invertir un bit aleatorio de un hijo.
10. Decodificar hijos a valores reales.
11. Actualizar población (Unir $N/2$ padres + $N/2$ hijos).
12. FIN PARA
13. Retornar el individuo con mayor evaluación.
14. FIN

```
In [2]: def optimizar_rastrigin(num_poblacion=512, generaciones=100):
    start_time = time.time()

    # Parámetros de la función y dominio
    A = 10
    xmin, xmax = -5.12, 5.12
    tpaso = 0.001

    # Cálculo de elementos y número de bits
    elementos = (xmax - xmin) / tpaso + 1
    nbits = int(np.ceil(np.log2(elementos)))

    # Población inicial
    x1 = np.random.randint(0, 2 ** nbits, num_poblacion)
    x2 = np.random.randint(0, 2 ** nbits, num_poblacion)
    x1real = (xmax - xmin) * x1 / (2 ** nbits - 1) + xmin
    x2real = (xmax - xmin) * x2 / (2 ** nbits - 1) + xmin

    yprom = np.zeros(generaciones)

    for i in range(generaciones):
        # 1. Evaluación
        y = 2 * A + x1real**2 + x2real**2 - A * np.cos(2 * np.pi * x1real) -
        yprom[i] = np.mean(y)

        # 2. Selección (Ordenamos y tomamos la mitad superior para MAXIMIZAR)
        indices_ordenados = np.argsort(y)
        x1_ordenado, x2_ordenado = x1[indices_ordenados], x2[indices_ordenados]
        x1real_ordenado, x2real_ordenado = x1real[indices_ordenados], x2real[indices_ordenados]

        mejores_x1, mejores_x2 = x1_ordenado[num_poblacion//2:], x2_ordenado[num_poblacion//2:]
        mejores_x1real, mejores_x2real = x1real_ordenado[num_poblacion//2:], x2real_ordenado[num_poblacion//2:]

        # 3. Conversión a binario (Cromosoma unido)
        padresbin = np.zeros((num_poblacion//2, 2 * nbits), dtype=int)
        for idx in range(num_poblacion//2):
            v1, v2 = mejores_x1[idx], mejores_x2[idx]
            for j in range(nbits):
                padresbin[idx, j] = (int(v1) >> j) & 1
                padresbin[idx, nbits + j] = (int(v2) >> j) & 1
```

```

# 4. Cruzamiento (Un punto)
hijobin = np.zeros_like(padresbin)
for k in range(num_poblacion // 4):
    m = np.random.randint(1, (2 * nbits) - 1)
    padre1, padre2 = padresbin[2*k], padresbin[2*k+1]
    hijobin[2*k] = np.concatenate((padre1[:m], padre2[m:]))
    hijobin[2*k+1] = np.concatenate((padre2[:m], padre1[m:]))

# 5. Mutación (Probabilidad global del 10%)
if np.random.rand() > 0.90:
    nhijo = np.random.randint(0, num_poblacion//2)
    nbit = np.random.randint(0, 2 * nbits)
    hijobin[nhijo, nbit] = 1 - hijobin[nhijo, nbit]

# 6. Decodificación de hijos
hijoent1, hijoent2 = np.zeros(num_poblacion // 2), np.zeros(num_poblacion // 2)
for idx in range(num_poblacion // 2):
    val1, val2 = 0, 0
    for j in range(nbits):
        val1 += hijobin[idx, j] * (2**j)
        val2 += hijobin[idx, nbits + j] * (2**j)
    hijoent1[idx], hijoent2[idx] = val1, val2

hijoreal1 = (xmax - xmin) * hijoent1 / (2 ** nbits - 1) + xmin
hijoreal2 = (xmax - xmin) * hijoent2 / (2 ** nbits - 1) + xmin

# 7. Actualizar población
x1 = np.concatenate((mejores_x1, hijoent1))
x2 = np.concatenate((mejores_x2, hijoent2))
x1real = np.concatenate((mejores_x1real, hijoreal1))
x2real = np.concatenate((mejores_x2real, hijoreal2))

# Evaluación final
y_final = 2 * A + x1real**2 + x2real**2 - A * np.cos(2 * np.pi * x1real)
val_max = np.max(y_final)
ind_max = np.argmax(y_final)

tiempo_exec = time.time() - start_time

return yprom, val_max, x1real[ind_max], x2real[ind_max], tiempo_exec

```

3. Pruebas y Resultados

```

In [3]: # Casos de prueba
casos = [
    {"pob": 64, "gen": 50},
    {"pob": 100, "gen": 100},
    {"pob": 512, "gen": 100}, # Original
    {"pob": 512, "gen": 300},
    {"pob": 1000, "gen": 200},
    {"pob": 2000, "gen": 500}
]

```

```

resultados = []
fig, axs = plt.subplots(2, 3, figsize=(15, 8))
fig.suptitle('Desempeño Promedio por Caso de Prueba', fontsize=16)

for idx, caso in enumerate(casos):
    yprom, max_val, x1_opt, x2_opt, t_exec = optimizar_rastrigin(caso["pob"])

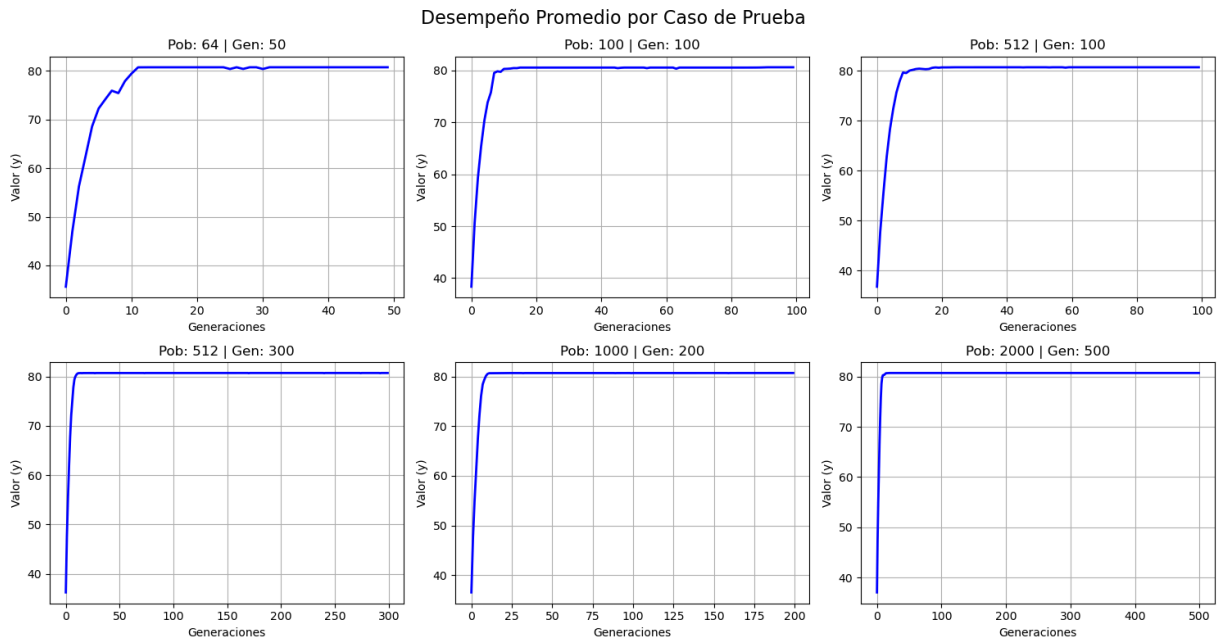
    # Guardar resultados
    resultados.append({
        "Población": caso["pob"],
        "Generaciones": caso["gen"],
        "Max Valor Obtenido": round(max_val, 4),
        "x1": round(x1_opt, 4),
        "x2": round(x2_opt, 4),
        "Tiempo (s)": round(t_exec, 4)
    })

    # Graficar
    ax = axs[idx//3, idx%3]
    ax.plot(yprom, color='blue', linewidth=2)
    ax.set_title(f'Pob: {caso["pob"]} | Gen: {caso["gen"]}')
    ax.set_xlabel('Generaciones')
    ax.set_ylabel('Valor (y)')
    ax.grid(True)

plt.tight_layout()
plt.show()

# Mostrar Tabla
df_resultados = pd.DataFrame(resultados)
display(df_resultados)

```



	Población	Generaciones	Max Valor Obtenido	x1	x2	Tiempo (s)
0	64	50	80.7027	4.5193	-4.5256	0.0308
1	100	100	80.6404	-4.5393	-4.5143	0.0670
2	512	100	80.7056	4.5243	-4.5212	0.3875
3	512	300	80.7030	4.5200	4.5200	1.8153
4	1000	200	80.7046	-4.5231	4.5262	1.9408
5	2000	500	80.7065	-4.5231	4.5225	7.8262

Análisis de Resultados de la Población Final

- **Óptimo real dentro del dominio:** La función Rastrigin clásica se minimiza en $(0, 0)$ con valor 0. Al maximizarla dentro del dominio acotado $[-5.12, 5.12]$, el valor más alto **no** se encuentra en las esquinas $(\pm 5.12, \pm 5.12)$, donde $f \approx 57.85$, sino cerca de $x_1, x_2 \approx \pm 4.523$, punto en el que el término $-10 \cos(2\pi x)$ alcanza su valor máximo ($+10$) mientras x^2 todavía es grande. Ahí la función alcanza su valor más alto dentro del dominio, ≈ 80.706 , que es justamente el valor al que convergen todos los experimentos.
- **Consistencia entre casos de prueba:** En todos los tamaños de población probados (desde $N = 64$ hasta $N = 2000$), el algoritmo converge de forma consistente a valores muy cercanos a 80.70, con x_1 y x_2 cercanos a ± 4.52 . Esto indica que, para este problema en particular, incluso poblaciones pequeñas son suficientes para encontrar el óptimo, ya que este se ubica relativamente cerca del centro del dominio y no en una región aislada difícil de alcanzar.
- **Costo computacional:** El principal efecto de incrementar la población y el número de generaciones no es una mejora sustancial en la calidad de la solución (que ya es casi óptima desde $N = 64$), sino el aumento del tiempo de ejecución, que crece de forma aproximadamente proporcional a $N \cdot G$ (de 0.03 s con $N = 64, G = 50$ a 7.83 s con $N = 2000, G = 500$).

Conclusiones

- El Algoritmo Genético con codificación binaria logró optimizar correctamente la función Rastrigin en dos dimensiones: en todos los casos de prueba encontró un valor muy cercano al máximo real del problema siendo este ≈ 80.70 , lo que confirma que el esquema de selección, cruce y mutación implementado es adecuado para este tipo de problema.
- El máximo real de la función dentro del dominio no está en las esquinas, sino en puntos intermedios ($x_1, x_2 \approx \pm 4.52$). Esto reforzó la importancia de verificar los

resultados del algoritmo contra un análisis matemático de la función, en lugar de asumir de antemano dónde se encuentra el óptimo.

- El tamaño de la población y el número de generaciones afectan principalmente el tiempo de ejecución, más que la calidad de la solución final: incluso con una población pequeña ($N = 64$) el algoritmo encontró una solución casi óptima, mientras que poblaciones grandes ($N = 2000$) tardaron mucho más tiempo sin lograr una mejora relevante.
- En general, el ejercicio confirma que los Algoritmos Genéticos son una herramienta efectiva para resolver problemas de optimización con múltiples óptimos locales, como Rastrigin, ya que combinan exploración (diversidad de la población) y explotación (selección de los mejores individuos) para converger hacia buenas soluciones sin necesidad de calcular derivadas ni conocer previamente la forma exacta de la función.
- Como trabajo futuro, sería interesante probar tasas de mutación más altas o variables, así como otros esquemas de selección y cruce, para evaluar el comportamiento del algoritmo en problemas donde el óptimo se encuentre en zonas del dominio más difíciles de alcanzar que en este caso.

Conclusiones personales

- **Dayanni:** Es una función muy útil para medir la capacidad y precisión de tu modelo de optimización, ya que es muy sensible a los parámetros, tiene como trampas en las que un modelo no tan bueno, fallaría. Un algoritmo que logra resolver Rastrigin exitosamente demuestra un excelente balance entre explorar todo el espacio disponible y refinar la mejor zona una vez que la ha encontrado.
- **Estephany:** Una vez terminada esta práctica entiendo aún más como a cada algoritmo en especial este, que realizamos anteriormente puede cumplir con las restricciones y especificaciones que el problema requiera, por ejemplo ; maximizar la función (RASTRIGIN) o bien minimizar de acuerdo a los parámetros y/o restricciones que deseemos con el objetivo de llegar al mejor óptimo global.
- **Fernanda:** A nivel personal, esta práctica me ayudó a entender mejor cómo la selección, el cruce y la mutación trabajan juntos para mejorar una población, y a no asumir dónde está el óptimo de un problema sin comprobarlo matemáticamente.
- **Valeria:** En lo personal, esta práctica me ayudó a entender mejor cómo funciona un algoritmo genético y cómo influye el tamaño de la población y las generaciones en los resultados. Pude observar que aumentar estos parámetros ayuda a acercarse al óptimo global, aunque también aumenta el tiempo de ejecución. En general, aprendí que es importante encontrar un equilibrio entre la calidad de la solución y el costo computacional.